

Relatório Técnico — Núcleo Cognitivo da Aurora Siger

Núcleo Cognitivo da Aurora Siger (NCAS) — Cap 1, Fase 5

1. Sumário executivo

O NCAS é um protótipo de sistema de apoio à decisão para a colônia marciana da missão Aurora Siger.

Ele faz três coisas. Guarda os registros operacionais da base em duas camadas de persistência, uma em JSON e outra em texto puro. Classifica alertas automaticamente, aplicando regras booleanas simplificadas por teoremas de álgebra. E monta prompts estruturados que orientam a geração de respostas padronizadas ao Centro de Comando, com a resposta produzida localmente, sem chamada a nenhum modelo de linguagem.

São 540 linhas de Python em arquivo único, sem dependência externa: nove seções, trinta funções e um menu de terminal com dez opções. Os treze módulos da colônia vieram da Fase 4, com os mesmos nomes e atributos.

Resultado principal: o sistema mostra em execução que uma classificação automatizada por regras pode ser rápida sem deixar de ser auditável. Cada alerta analisado exibe na tela a expressão booleana que produziu o resultado, os valores de entrada que a alimentaram e a declaração de que a decisão final cabe à equipe humana do Centro de Comando.

2. O problema

A infraestrutura da Aurora Siger cresceu e passou a produzir mais registro do que a equipe técnica consegue ler. Chegam alertas operacionais dos módulos, logs de funcionamento, medições de consumo energético, solicitações da tripulação e relatórios técnicos.

Falta de dado não é o problema. O problema é que dado guardado sem organização não vira conhecimento: ninguém localiza um registro antigo, ninguém separa rapidamente o que é crítico do que é rotina, e a validação das condições de operação continua sendo feita de cabeça, por quem estiver de plantão.

O NCAS foi construído para atacar esses quatro pontos.

3. Linha de raciocínio do projeto

As seções seguintes registram as decisões de projeto na ordem em que foram tomadas, com o motivo de cada uma. Não descrevem o que o sistema faz, e sim por que ele ficou assim.

3.1 Reaproveitar a colônia da Fase 4

A Fase 4 já tinha modelado a Aurora Siger como um grafo de treze módulos: Núcleo Energético, Suporte a Vida Central, Habitats A, B e C, Estufa Hidropônica, Campos Solares Norte e Sul,

Reciclagem de Água, Centro de Comando, Laboratório de Pesquisa, Estação de Armazenamento e Hangar de Veículos. Cada um com tipo, consumo em kW, status e prioridade operacional.

Inventar módulos novos teria sido mais rápido. A opção foi importar os treze existentes, descartando apenas os campos de grafo que não se aplicam aqui: `conexoes`, `peso` e `geracao_kw`.

O ganho acabou sendo maior que a continuidade narrativa. O campo `prioridade_operacional`, definido lá atrás para calcular pontos de articulação da rede, virou uma das três variáveis da regra de criticidade desta fase.

3.2 Duas camadas de persistência, com critério explícito

A escolha entre texto e JSON seguiu o modo de acesso ao dado, e não o seu conteúdo:

ARQUIVO	CONTEÚDO	POR QUÊ
<code>dados_colonia.json</code>	Módulos e alertas, registros com campos fixos	O programa precisa consultar e filtrar por campo. JSON mapeia direto para dict e list em Python via <code>json.load</code> e <code>json.dump</code> , sem código de parsing.
<code>registros_colonia.txt</code>	Log de interações, uma linha por evento	Eventos cronológicos, lidos por humanos, que só crescem e nunca são consultados por campo. Texto em modo <code>append</code> resolve, e resolve mais barato.

A regra que resume o critério cabe em uma frase: dado que o programa vai reler e filtrar fica em JSON, dado que só cresce e é lido por gente fica em texto.

3.3 O que caracteriza um alerta

Cada alerta guarda os campos que o enunciado sugere: módulo de origem, tipo de ocorrência, prioridade, data e mensagem. Guarda também dois booleanos, `falha_detectada` e `consumo_levado`.

Esses dois foram definidos antes da regra de criticidade, por engenharia reversa a partir dela. São as entradas verificáveis que a regra consome.

3.4 Duas regras lógicas em vez de uma

O enunciado pede pelo menos uma regra booleana, simplificada pelo teorema de Simplificação *ou* pelo de De Morgan. O critério de avaliação, porém, fala em "simplificação e De Morgan". Implementar as duas custou cerca de dez linhas de código a mais e cobre o critério inteiro.

As duas não são exercícios paralelos. Elas atuam em sequência sobre a mesma operação: o sistema verifica primeiro se o operador pode ver o alerta, e só então classifica aquele alerta.

```
REGRA B (De Morgan) - roda primeiro, e uma questao de autorizacao
    Original:  BLOQUEAR = NOT (AUTORIZADO AND MODULO_ATIVO)
    Simplif.:  BLOQUEAR = (NOT AUTORIZADO) OR (NOT MODULO_ATIVO)

REGRA A (distributiva) - roda depois, e uma questao de classificacao
    Original:  CRITICO = (FALHA AND MOD_CRITICO) OR (FALHA AND CONSUMO_ALTO)
    Simplif.:  CRITICO = FALHA AND (MOD_CRITICO OR CONSUMO_ALTO)
```

A ordem é deliberada e vem de um princípio de segurança: verificar permissão antes de processar. Com o acesso bloqueado, o sistema não calcula criticidade nem exibe o conteúdo do alerta a quem não pode consultá-lo.

3.5 Por que a simplificação de De Morgan valeu a pena

A Regra B usa duas operações lógicas nas duas formas. Não houve redução alguma, e ainda assim a troca compensou, por um motivo que não tem a ver com desempenho.

Na forma original o resultado é um veredito único, sem causa. Na forma de De Morgan as duas causas ficam escritas separadamente, ligadas por um OR, e o sistema pode testar cada uma antes de responder:

```
ACESSO NEGADO: o modulo Campo Solar Sul esta INATIVO.
ACESSO NEGADO: usuario sem autorizacao.
```

Nenhuma dessas duas mensagens seria possível com a expressão original. A simplificação mudou o que o sistema consegue comunicar ao operador, e isso é verificável rodando a opção 3 do menu sobre o alerta 6 e sobre o alerta 1 sem autorização.

3.6 Por que o exemplo do enunciado não foi utilizado

O material da fase traz como exemplo conceitual $ALERTA = (FALHA \text{ AND } CRITICO) \text{ OR } (FALHA \text{ AND } NOT \text{ CRITICO})$, que simplifica até $ALERTA = FALHA$. É claro do ponto de vista didático, mas a regra some no processo: passa a ignorar todas as outras variáveis.

A Regra A implementada aqui tem a mesma estrutura algébrica e demonstra o mesmo teorema, só que preserva três entradas e continua separando casos.

3.7 A classificação não confia no rótulo do operador

Existem duas noções de prioridade no sistema, mantidas deliberadamente separadas.

- prioridade é o rótulo de urgência que o operador atribuiu ao registrar o alerta. Subjetivo, sujeito a pressa e a viés.

- `prioridade_operacional` é a importância do módulo para a sobrevivência da colônia, cadastrada na Fase 4. Objetiva e estável.

A criticidade usa apenas a segunda. A urgência, que é comunicação e não classificação, pode considerar a primeira. Um sistema que classificasse pelo rótulo digitado estaria apenas devolvendo o julgamento de quem digitou, com aparência de objetividade.

3.8 Respostas montadas sem esconder a lógica

A integração com API é opcional no enunciado, e este projeto não a utiliza. O risco de uma simulação local é produzir uma fachada: textos fixos guardados no código e exibidos como se fossem análise. Duas decisões evitam isso.

- **Os prompts são funções.** Cada um recebe o alerta e o módulo correspondente e injeta os dados no molde. Troque o alerta e o prompt gerado muda.
- **As respostas vêm das regras.** A criticidade sai de `eh_critico()`, a urgência de `classificar_urgencia()` e a ação de `acao_recomendada()`. Padronizada mesmo, só a redação.

O prompt *chain-of-thought* deixa isso claro. O passo a passo exibido na tela é preenchido com os valores que o sistema calculou de fato, não com uma narrativa montada depois para justificar o resultado.

3.9 Arquitetura em camadas

O código está dividido em nove seções, da mais baixa para a mais alta, de modo que cada camada só depende das anteriores:

SEÇÃO	RESPONSABILIDADE
[1] Constantes	Caminhos dos arquivos, definidos em um único lugar
[2] Camada JSON	<code>carregar_dados</code> e <code>salvar_dados</code>
[3] Camada texto	<code>registrar_log</code> , <code>ler_logs</code> e <code>exibir_logs</code> , com <code>read</code> , <code>readline</code> , <code>readlines</code> e <code>writelines</code>
[4] Registros	Busca, cadastro e listagem de módulos e alertas
[5] Regras lógicas	As duas regras em ambas as formas, mais a tabela-verdade
[6] Prompts	Os quatro prompts estruturados
[7] Geração de respostas	Respostas operacionais padronizadas e análise de refinamento
[8] Análise de alerta	Funcionalidade-vitrine, que integra de [2] a [7]
[9] Menu	Interface de terminal e ponto de entrada

As regras da seção [5] recebem apenas `True` e `False`. Não conhecem alertas nem módulos, e é esse isolamento que permite testá-las por tabela-verdade sem carregar dado nenhum. A tradução entre os dados e os booleanos mora em uma função-ponte separada, `condicoes_do_alerta()`.

4. Resultados

ENTREGA	SITUAÇÃO
Persistência em JSON e texto	13 módulos e 7 alertas em JSON; log preservado entre execuções
Métodos de arquivo exigidos	read, readline, readlines e writelines, cada um onde é a ferramenta certa
Regras booleanas	Duas regras, dois teoremas, ambas as formas no código
Prova de equivalência	12 linhas de tabela-verdade, executáveis pela opção 4 do menu
Prompts estruturados	Zero-shot, few-shot, chain-of-thought e structured output
Simulação de resposta estruturada	Respostas padronizadas derivadas das regras, sem API externa
Análise de otimização	Comparação entre prompt vago e prompt refinado, opção 7
Interface	Menu de 10 opções com validação de entrada

5. Casos de demonstração

A base foi montada para que cada caminho do código tivesse um caso real que o exercitasse. Três alertas merecem destaque.

ALERTA	MÓDULO	O QUE DEMONSTRA
#1	Suporte a Vida Central	Falha em módulo de prioridade CRÍTICA. Sai como CRÍTICO, urgência URGENTE, ação imediata.
#3	Laboratório de Pesquisa	Tem falha detectada e mesmo assim não é crítico: o módulo é MÉDIA e não há consumo elevado. Prova que a regra separa casos em vez de carimbar tudo.
#6	Campo Solar Sul	Único módulo INATIVO da colônia. Exercita o bloqueio por módulo fora de operação, que só é distinguível graças à forma de De Morgan.

O #3 é o mais importante dos três para julgar o sistema. Um núcleo cognitivo que classificasse toda falha como crítica soaria alarme o tempo todo, e em pouco tempo a equipe passaria a ignorá-lo.

6. Limitações conhecidas

- **Sem modelo de linguagem integrado.** As respostas são montadas localmente por templates e regras booleanas. O sistema demonstra a estrutura dos prompts e a lógica de classificação, mas não executa geração textual por LLM.
- **Busca linear.** `buscar_modulo()` e `buscar_alerta()` percorrem a lista inteira. Com treze módulos isso é irrelevante; em escala maior, um dicionário indexado por chave seria o passo

seguinte.

- **Sem controle de concorrência.** Duas execuções simultâneas gravando no mesmo JSON poderiam sobrescrever uma à outra. Aceitável para um terminal de operador único.
- **Autorização declarada, não verificada.** O sistema pergunta ao operador se ele está autorizado em vez de consultar um cadastro de credenciais. A regra booleana está correta; a fonte do dado é que foi simplificada.
- **Limiares fixos.** Os critérios de urgência estão no código. Em operação real seriam parâmetros configuráveis, revisados periodicamente.

7. Ética e responsabilidade

No NCAS, as respostas são montadas por templates e por regras que a equipe escreveu. Os critérios de classificação, portanto, precisam ficar documentados, e precisam poder ser revisados e auditados por quem opera a colônia.

Em sistemas que usam modelos treinados sobre dados, esse cuidado tem de ser maior, porque os dados carregam vieses históricos que ninguém escreveu de propósito. Um sistema de priorização de atendimento pode acabar despriorizando setores, turnos ou grupos da tripulação sem que exista uma linha de código dizendo isso.

Equipes diversas identificam vieses que equipes homogêneas não enxergam, porque reconhecem os padrões que as afetam. A composição de quem constrói o sistema é uma variável técnica, não apenas social.

Quatro decisões concretas do projeto respondem a isso:

- Classificar sobre fatos verificáveis, a prioridade cadastrada do módulo, e não sobre o rótulo subjetivo que o operador digitou.
- Imprimir a expressão booleana e os valores de entrada em cada análise. Classificação que se explica é classificação que se pode contestar.
- Usar linguagem técnica e neutra nas respostas, referida a módulos e ocorrências.
- Declarar, ao final de cada análise, que a decisão final cabe à equipe do Centro de Comando.

O princípio que orienta o projeto: **o sistema sugere, a pessoa decide.** Automatizar a análise é aceitável. Automatizar a responsabilidade não é.

8. Conclusão

O NCAS reúne em um protótipo funcional os conteúdos das cinco disciplinas da fase: manipulação de arquivos e JSON em Python, álgebra booleana com simplificação por teoremas, engenharia de prompts e modelos de linguagem, memória e fluxo de dados, e responsabilidade social no uso de sistemas automatizados.

O que liga as partes não é a justaposição. A mesma expressão booleana que o Python avalia é a que o prompt *chain-of-thought* pede para ser percorrida passo a passo. A simplificação de De Morgan não ficou no papel: é ela que permite ao sistema informar a causa de um bloqueio. E a reflexão ética não é um anexo, é uma linha que o programa imprime toda vez que classifica um

alerta.